Lecture 9 · Wed 10/28/2026

CS160 Compilers — Fall 2026

Type Checking.

Yu Feng

Where the checker sits.



semantic analysis

the pass after parsing that asks what the program *means*, not how it is written. The first pass that can reject a program it could read perfectly well. We call it *the checker*.

Monday ended with `print(f(1, 2))` for a one-parameter `f` compiling and printing 1. The parser cannot see anything wrong with it, and clang cannot either. Today is the pass that can. First, why such a pass is worth having.

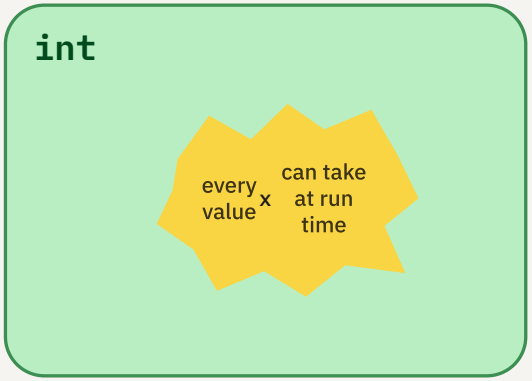
We would like to prove programs correct. We cannot.

Programs crash, or compute the wrong thing. We would like to *prove*, before running, that a program has no error.

But no algorithm can do that for every program: for any interesting property, there is no checker that is right about all programs. This is a theorem (Rice, 1953), not a limitation of today's tools.

So: give up? No. **Check something simpler than the program.**

Check an abstraction of the program instead.



concrete value

what the program actually computes when it runs: `10`, `-3`, `True`

abstract value

a description that stands for many concrete values at once, and throws the rest away

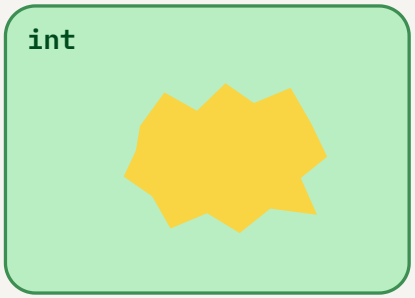
abstraction

replacing the program's concrete values by abstract ones, and reasoning about those.

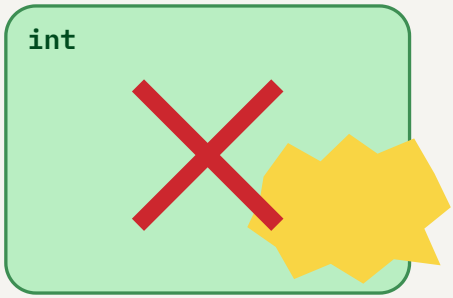
```
x: int = 10          concrete:  x is 10          abstract:  x is an int
y: int = x + 1       concrete:  y is 11          abstract:  y is an int
```

We cannot know what every program computes. We *can* know, for every program, what *kind* of thing each expression computes. That kind is its **type**, and types are the abstraction this course uses.

The abstraction must be sound.



sound: yellow inside green



not sound: yellow leaves green

sound

the abstract value always *contains* the concrete value: whatever the program really computes falls inside what the type says. `int` is sound for `10`; `bool` would not be

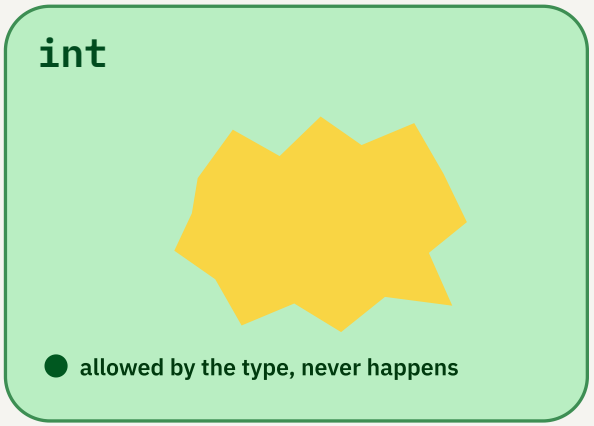
overapproximate

contain, possibly with room to spare — the type may allow more than the program will ever do, never less.

Why it matters: if the abstraction is sound and reports *no error*, the real program has no error of that kind either. The checker says `x + 1` is an `int`; because the abstraction is sound, running it produces an `int` — never a crash from adding a `bool` to a number.

A type checker is exactly this: run the program on types instead of values, and reject any program where the types do not fit together.

What the abstraction costs.



false positive

a program the checker rejects that would in fact have run without error.

falsepos.py · verified, rejected by typecheck.py

```
b: bool = True
x: int = 0
if False:
    x = b + 1        # never runs
                     # rejected:  line 4: expected int, found bool
```

The type says b could be any `bool`, and `bool + 1` has no rule, so the line is rejected although it never executes. That is the price of checking the abstraction instead of the program: some fine programs are refused. The trade is worth it: every program it accepts is guaranteed free of type errors.

Where the types come from.

type checking

the programmer writes the types (`x: int`, `def f(a: int) -> int`) and the checker verifies that every use is consistent with them. C, Java, ChocoPy

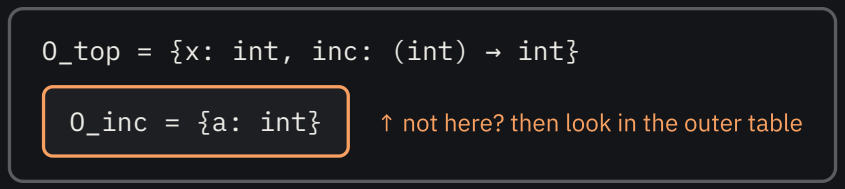
type inference

the programmer writes none; the compiler works out the best type of every expression and complains if they cannot be made to fit. ML, OCaml, Haskell.

ChocoPy checks: every variable and every parameter carries a declared type, so the checker never has to guess — it only has to *verify*. That is what the rest of today builds.

What the checker knows.

```
O  =  { x: int,  inc: (int) → int }        a table: every declared name, and its type
R  =  int                                   the return type of the function we are inside  (⊥ at top level: none)
```



scope
one such table for one region of the program. A function body gets its own table, with the top-level table behind it; looking a name up tries the inner table first, then the outer

symbol table
the record of every declared name and what it stands for — in Lecture 3 a *place*, today a *type*. The manual calls it O

R
the return type the enclosing function promised, so that a `return` can be checked against it.

Two things, and the rules need nothing else: what every name is, and what the current function must return.

The types of ChocoPy.

manual §2.4

```
int       bool      str              the three primitive types
[T]       a list whose elements have type T       e.g.  [int]
object    the type every value belongs to;  no ChocoPy program this quarter writes it
<None>    the type of the one value None.  Not writable: None is a value, <None> is its type
```

This quarter's subset uses `int`, `bool` and `<None>`; `str` and `[T]` arrive with PA4, and no rule changes.

One fact to carry: `None` may be stored into a list or `object` variable, never into an `int`, `bool` or `str` — those three have no ‘empty’ value. Slide 13 says where this is used; Monday gives the full rule.

A judgment, and the shape of a rule.

manual §5.2

```
O, M, C, R ⊢ e : T        the manual's judgment.  M: class members;  C: the current class.
O, R ⊢ e : T              ours: no classes, so M is empty and C is ⊥.  Nothing else differs.
```

$$\frac{\text{premises}}{\text{conclusion}}\ [\text{name}]$$

judgment
a claim that is true or false: *in environment O, inside a function returning R, e has type T*

typing rule
a bar: *if* the **premises** above it hold, *then* the **conclusion** below it holds. A rule with no premises is an **axiom**

⊢ · :
⊢ reads *proves*; : reads *has type*

side condition
a fact written beside a rule — looked up, not derived.

These rules are the definition of ChocoPy's types; nothing else says what the language means. Slide 5's ‘run the program on types’ means: apply these rules to the tree.

Rules for expressions: literals, variables, operators.

$$\frac{}{O, R \vdash n : \text{int}}\ [\text{int}] \qquad \frac{}{O, R \vdash \text{True} : \text{bool}}\ [\text{bool-true}] \qquad \frac{}{O, R \vdash \text{False} : \text{bool}}\ [\text{bool-false}]$$

$$\frac{O(\text{id}) = T, \text{ where } T \text{ is not a function type}}{O, R \vdash \text{id} : T}\ [\text{var-read}] \qquad \frac{O, R \vdash e1 : \text{int} \quad O, R \vdash e2 : \text{int}}{O, R \vdash e1 \text{ op } e2 : \text{int}}\ [\text{arith}]$$

$$\text{op} \in \{+, -, *, //, \%\}$$

$$\frac{O, R \vdash e : \text{int}}{O, R \vdash -e : \text{int}}\ [\text{negate}] \qquad \frac{O, R \vdash e : \text{bool}}{O, R \vdash \text{not } e : \text{bool}}\ [\text{not}] \qquad \frac{O, R \vdash e1 : \text{bool} \quad O, R \vdash e2 : \text{bool}}{O, R \vdash e1 \text{ and } e2 : \text{bool}}\ [\text{and}], [\text{or}]$$

`O(id) = T`
look `id` up in O. If it is not there, `[var-read]` cannot fire — *undeclared name*

∈
is one of.

Read `[arith]` aloud: *if e1 is an int and e2 is an int, then e1 op e2 is an int.* Slide 5's rule for `x + 1`, written down. `[not]`, `[and]`, `[or]`: `bool` in, `bool` out — `if 1 and x:` is not ChocoPy.

Rules for expressions: comparison and function call.

```
O, R ⊢ e1 : int    O, R ⊢ e2 : int                O, R ⊢ e1 : bool    O, R ⊢ e2 : bool
--------------------------------- [int-compare]  ---------------------------------- [bool-compare]
O, R ⊢ e1 ./ e2 : bool                           O, R ⊢ e1 ./ e2 : bool
   ./ ∈ {<, <=, >, >=, ==, !=}                      ./ ∈ {==, !=}

O(f) = (T1, ..., Tn) → T0        O, R ⊢ e1 : T1   ...   O, R ⊢ en : Tn
------------------------------------------------------------------- [invoke]
O, R ⊢ f(e1, ..., en) : T0
```

`./`
whichever comparison operator was written

two compare rules
`==` on two `int`s is `[int-compare]`; on two `bool`s, `[bool-compare]`; nothing types `1 == True`

function type `(T1, ..., Tn) → T0`
parameter types, then the result type — what `O` holds for a function name.

`[invoke]`: exactly *n* argument premises, one per parameter, each of the parameter's type. Monday's `f(1, 2)` dies here — `f` has one parameter and the call has two arguments.

Rules for statements.

A statement has no type. Its judgment is `O, R ⊢ s` — *statement s is well-formed in O* — and a declaration also hands back the new environment, written `=> O[x ↦ T]`.

```
O, R ⊢ e : T1   T1 ≤ₐ T   x ∉ dom(O)          O(x) = T   O, R ⊢ e : T1   T1 ≤ₐ T
----------------------------------- [var-init]  ------------------------------ [var-assign-stmt]
O, R ⊢ x: T = e   => O[x ↦ T]                    O, R ⊢ x = e

O, R ⊢ c : bool   O, R ⊢ s1 … sn   O, R ⊢ t1 … tm      O, R ⊢ c : bool   O, R ⊢ s1 … sn
---------------------------------------- [if-elif-else]  ---------------------------- [while]
O, R ⊢ if c: s1 … sn else: t1 … tm                        O, R ⊢ while c: s1 … sn

O, R ⊢ e : T   T ≤ₐ R              O, R ⊢ e : T
---------------------- [return-e]  -------------- [expr-stmt]
O, R ⊢ return e                    O, R ⊢ e
```

well-formed

obeys the rules; the word for a statement, since it has no type to report

`T1 ≤ₐ T` · assignment compatible

a value of type `T1` may be stored where a `T` is expected. For us: the same type, or `None` stored into a non-primitive (slide 9). Every rule that *stores* a value — initialize, assign, return — asks this, never 'the same type?'

`=>`

and afterwards the environment is: the one rule that changes O

`O[x ↦ T]`

O plus one binding: after the declaration, `x` has type `T`

`x ∉ dom(O)`

`x` is not already declared here

`⊥`

`R` at top level: no enclosing function, so `return` has nothing to return to.

`[if-elif-else]`, `[while]`: the condition must be a `bool`. `[return-e]`: the value must be storable as the function's result type. `print(e)` has no rule of its own: it is `[expr-stmt]` over `[invoke]`, `print` being a predefined function.

Functions.

$$\frac{O(f) = (T1, \ldots, Tn) \to T \qquad O' = O \oplus \{x1: T1, \ldots, xn: Tn\} \qquad O', T \vdash s1 \ldots sk}{O, \bot \vdash \texttt{def f(x1: T1, ..., xn: Tn) -> T: s1 … sk}}\ [\text{func-def}]$$

⊕

O with these bindings added — slide 13's `O[x ↦ T]` for all the parameters at once

O'

a *new* environment: the parameters are visible in the body and nowhere else

O', T ⊢

the body is checked with `R = T`: inside `f`, every `return e` must produce something storable as `T`.

Two conditions the manual states in words rather than as rules. *Return completeness* (§2.8.5): a function returning `int`, `str` or `bool` must `return` a value on every path — a walk over the statement list, where a `while` never counts.

Where does `O(f)` *come from?* Pass 1 reads every `def` header into `O_top`; pass 2 checks every body and every top-level statement. Without pass 1, a function could not call itself.

A program, checked · 1/4

The program, and pass 1.

run.py · verified: prints True

```
x: int = 3                          # line 1
def inc(a: int) -> int:             # line 2
    return a + 1                    # line 3
print(inc(x) < 5)                   # line 4
```

```
pass 1, the headers:    O_top = { x: int,  inc: (int) → int,  print: (object) → <None> }      R = ⊥
pass 2, the body:       check line 1, then the def (lines 2–3), then line 4, each under O_top
```

Three statements to check. Each one is a derivation whose bottom line is `O_top, ⊥ ⊢ statement`; the next three slides are those derivations. Monday runs this same program.

Line 1 and the function.

line 1, by [var-init]

```
------------- [int]
O, ⊥ ⊢ 3 : int        int ≤ₐ int        x ∉ dom(O)
------------------------------------------------- [var-init]
O, ⊥ ⊢ x: int = 3    => O[x ↦ int]
```

lines 2–3, by [func-def] · O' = O ⊕ {a: int}, body checked with R = int

```
O'(a) = int
---------------- [var-read]   ------------- [int]
O', int ⊢ a : int             O', int ⊢ 1 : int
------------------------------------------- [arith]
O', int ⊢ a + 1 : int                        int ≤ₐ int
------------------------------------------------------------------ [return-e]
O', int ⊢ return a + 1
------------------------------------------------------------------------------ [func-def]
O, ⊥ ⊢ def inc(a: int) -> int: return a + 1
```

derivation
the finished stack of bars: a proof that the bottom judgment holds, each step justified by a named rule

well-typed
a derivation exists.

Read each one from the bottom up: the statement, the rule whose conclusion has its shape, the smaller obligations that rule leaves, down to axioms and lookups. The function's body is checked in `O'`, where `a` is known, with `R = int` so the `return` has something to match.

A program, checked · 3/4

Line 4, as one tree.

O = O_top, with x now bound

```
                        O(x) = int
                        ---------------- [var-read]
O(inc) = (int) → int    O, ⊥ ⊢ x : int
---------------------------------------- [invoke]   ------------- [int]
O, ⊥ ⊢ inc(x) : int                                 O, ⊥ ⊢ 5 : int
------------------------------------------------------------------------ [int-compare]
O(print) = (object) → <None>        O, ⊥ ⊢ inc(x) < 5 : bool
------------------------------------------------------------------------ [invoke]
O, ⊥ ⊢ print(inc(x) < 5) : <None>
------------------------------------------------------------------------ [expr-stmt]
O, ⊥ ⊢ print(inc(x) < 5)
```

Six bars, one per node of the AST. The root is the statement; the leaves are a lookup and a literal. Every expression in the program now has a type written on it — `inc(x)` is `int`, the comparison is `bool` — and codegen will read those types instead of working them out again (Monday).

The same program with one line broken.

run_bad.py

```
x: int = 3
def inc(a: int) -> int:
    return a + 1
print(inc(x) < True)              # line 4:  5 became True
```

```
O, ⊥ ⊢ inc(x) : int          O, ⊥ ⊢ True : bool
-------------------------------------------------  [int-compare] needs bool : int.   Have bool.   STUCK.
O, ⊥ ⊢ inc(x) < True : ???                         [bool-compare] needs inc(x) : bool.  Have int.  STUCK.
```

ill-typed

no derivation exists: at some node, no rule's premises can be satisfied.

Both comparison rules get stuck on the same node, so there is no tree and the program is rejected: `line 4: cannot compare int with bool` (verified). The stuck node knows its line number; that is the whole error message.

Monday; PA3.

Monday: first the checker's code — one `case` per rule — and the rules for `None` and lists that PA4 needs; then the yellow half of slide 5's picture: the rules for what a program *does* when it runs, in the same bar-and-premise shape, on this same program.

PA3 (due Fri 10/30): functions plus this checker; report the *first* error's line number. The rules and their names are the manual's §5.